

1 Adversarial Models in Cryptographic Systems

The security of a cryptographic system is defined with respect to the behavior and capabilities of the entities involved in the system. An entity may be trusted, semi-honest, or malicious depending on the assumptions made about its behavior.

An *adversarial model* specifies which entities may be compromised and what an adversary can cause a compromised entity to do. Thus, the adversarial model is not a property of the functionality itself. Rather, it describes the behavior and capabilities attributed to the entities participating in the system.

1.1 Trusted Entity

A *trusted entity* is an entity that is assumed to behave exactly according to the prescribed protocol and is not compromised by the adversary.

A trusted entity does not intentionally disclose protected information, modify protocol messages, or deviate from the specified computation. Consequently, no adversarial behavior needs to be considered for such an entity.

The assumption that an entity is trusted is the strongest assumption about its behavior and generally results in the simplest protocol design.

1.2 Semi-Honest Entity

A *semi-honest entity*, also called an *honest-but-curious entity*, follows the prescribed protocol correctly but may attempt to learn additional information from everything available to it during the execution.

A semi-honest entity does not modify messages, provide malformed inputs, or deliberately deviate from the protocol. However, it may store and analyze its input, received messages, intermediate values, and other information available to it in an attempt to infer private information.

Thus, security against a semi-honest entity requires that the information available to that entity during an honest execution does not reveal unauthorized information.

1.3 Malicious Entity

A *malicious entity* may arbitrarily deviate from the prescribed protocol. It may provide incorrect inputs, modify or omit messages, send inconsistent information to different entities, or otherwise attempt to violate the correctness, privacy, or other security properties of the protocol.

Therefore, a malicious entity represents a significantly stronger adversarial capability than a semi-honest entity.

A protocol designed to tolerate malicious entities generally requires additional mechanisms to detect or prevent such deviations. Depending on the application, these may include commitments, zero-knowledge proofs, authentication, consistency checks, or verifiable secret sharing.

1.4 Adversarial Model of a System

A cryptographic system may contain many different entities, and each entity can potentially be assigned a different trust assumption. For example, a system may contain users, servers, a dealer, an authority, and communication infrastructure. Each of these entities may be assumed to be trusted, semi-honest, or malicious.

Therefore, defining an adversarial model requires specifying:

1. which entities may be compromised;
2. the behavioral model of each potentially compromised entity;
3. the number of entities that may be simultaneously compromised; and

4. the capabilities available to the adversary through the compromised entities.

For example, in an MPC system with parties $P_1, \dots, P_n$, one may assume that all parties are semi-honest, or that up to t parties may be malicious. These are different adversarial models and generally require different protocol designs.

Thus, *trusted*, *semi-honest*, and *malicious* describe the assumed behavior of individual entities. The adversarial model of the complete system specifies which of these assumptions applies to which entities and how many entities may be under adversarial control.

2 Designing a Cryptographically Secure Scheme

Designing a cryptographic scheme begins with a precise description of the system, its intended functionality, and the adversary against which security is required.

A useful way to organize the design process is to proceed through three fundamental steps:

1. Define the **system model**.
2. Define the **functionality and security goals**.
3. Define the **adversarial model**.

These three components determine what the system is expected to accomplish and what it must remain secure against.

2.1 Step 1: Define the System Model

The first step is to specify the entities involved in the system and their relationships.

We identify the participants, their inputs and outputs, the communication channels between them, and any additional entities such as servers, dealers, authorities, or external services.

For example, in an MPC system, the system model may consist of n parties $P_1, \dots, P_n$, where each party holds a private input and the parties communicate through a network to jointly compute a function.

The system model answers the question:

Who are the entities involved, and how do they interact?

2.2 Step 2: Define the Functionality

Once the system model is specified, we define what the system is supposed to accomplish.

Let the parties provide inputs $x_1, \dots, x_n$. The desired functionality specifies how these inputs should be transformed into outputs. For example, an MPC protocol may be required to compute $y = f(x_1, \dots, x_n)$ while keeping the individual inputs private.

The functionality should specify the legitimate outputs and the information that may be revealed to the participants.

The functionality answers the question:

What should the system compute or accomplish?

2.3 Step 3: Define the Adversarial Model

After specifying the system and its intended functionality, we define the adversarial model.

We determine which entities may be compromised and specify the capabilities available to the adversary. In particular, we decide whether a potentially compromised entity is modeled as trusted, semi-honest, or malicious, and how many entities may be compromised simultaneously.

The adversarial model answers the question:

What can go wrong, and what power does the adversary have over the entities it controls?

2.4 Effect of the Adversarial Model on Protocol Design

The choice of adversarial model directly affects the security requirements and the design of the cryptographic protocol.

For example, if an entity is assumed to be trusted, the protocol does not need to protect against deviations by that entity. If the same entity is modeled as semi-honest, the protocol must prevent it from learning unauthorized information from its protocol view. If it is modeled as malicious, the protocol must additionally tolerate arbitrary deviations by that entity.

Consequently, changing the adversarial model can change the cryptographic mechanisms required by the system. A construction that is secure when entities are assumed to be semi-honest may not remain secure when those entities are allowed to behave maliciously.

Thus, cryptographic protocol design can be viewed as the process of constructing a system that implements the desired functionality while satisfying precisely defined security requirements under a specified adversarial model.

In the following sections, we examine how different adversarial assumptions affect the system model, the security requirements, and ultimately the design of the cryptographic protocol.

3 Effect of the Adversarial Model on Additive Secret Sharing

We now examine how the choice of adversarial model affects the security and functionality of an additive secret-sharing scheme. Consider a trusted dealer who holds a secret s and distributes additive shares $x_1, \dots, x_n$ to the participants, where $x_1, \dots, x_{n-1}$ are chosen uniformly at random and $x_n = s - \sum_{i=1}^{n-1} x_i$.

The security and correctness of this construction depend not only on the sharing mechanism but also on the behavior of the dealer.

3.1 Adversarial Behavior of Dealer

1. Trusted Dealer:

If the dealer is trusted, it is assumed to follow the sharing algorithm correctly. It chooses the required random shares, computes the final share correctly, and distributes the appropriate share to each participant.

In this case, the additive sharing scheme provides its intended properties. The shares satisfy the required relation, and all participants can reconstruct the secret when the required shares are combined.

Thus, when the dealer is trusted, there is no need to consider malicious behavior by the dealer.

2. Semi-Honest Dealer:

Suppose the dealer is semi-honest. The dealer follows the prescribed sharing algorithm correctly but may try to obtain additional information from the information available to it.

Since the dealer already possesses the secret s and generates all the shares itself, it does not obtain any additional information about s by observing the shares. The dealer knows the complete sharing, including all x_i , but this does not provide information beyond the secret it already possesses.

Therefore, in this setting, the semi-honest behavior of the dealer does not affect the correctness or secrecy of the additive sharing scheme, provided that the dealer follows the sharing algorithm correctly.

3. Malicious Dealer:

The situation changes significantly when the dealer is malicious. A malicious dealer is allowed to deviate arbitrarily from the prescribed sharing algorithm.

For example, consider the n -party additive sharing construction. The dealer is required to choose the shares consistently so that their sum equals the secret s . A malicious dealer may fail to generate or distribute the shares according to this procedure.

For instance, the dealer may distribute shares that do not satisfy the required relation. In that case, when the participants add their shares, they may obtain a value different from s . More seriously, the dealer may distribute shares that are inconsistent with one another, making it impossible for the participants to reconstruct the intended secret correctly.

Thus, even though the additive secret-sharing mechanism itself provides secrecy against an unauthorized collection of shares, a malicious dealer can violate the *correctness* of the sharing process.

This example illustrates an important principle in cryptographic protocol design: a cryptographic construction may be secure under one adversarial model but require additional mechanisms when a different entity is allowed to behave maliciously.

In the additive secret-sharing example, the dealer is therefore a critical entity whose adversarial behavior directly affects the correctness of the system.

3.2 Adversarial Behavior of Participant

We now consider the effect of the adversarial model applied to the participants. Suppose the dealer correctly generates and distributes the additive shares $x_1, \dots, x_n$. The behavior of the participants can still affect the security and correctness of the overall system.

1. **Trusted Participants:** If all participants are trusted, they are assumed to use their shares exactly as prescribed by the protocol. In particular, they do not disclose their shares to unauthorized entities and do not deviate from the prescribed computation.

In this setting, the additive sharing scheme provides its intended security and correctness properties. The participants can combine their shares when reconstruction is required, and no additional protection against participant behavior is necessary.

2. **Semi-Honest Participants:** Suppose a participant is semi-honest. The participant follows the prescribed protocol correctly but may try to obtain additional information from its own share and everything else available to it.

For additive secret sharing, a single share x_i is statistically independent of the secret s . Therefore, a semi-honest participant holding only its own share does not obtain any additional information about s .

However, if multiple participants collude, their combined shares may provide additional information. In an n -out-of- n additive sharing scheme, any collection of fewer than n shares reveals no information about s , while all n shares are sufficient to reconstruct it.

Thus, in the semi-honest setting, the security of the scheme depends on the number of participants that the adversary can corrupt and whether their combined shares are sufficient to reconstruct the secret.

3. **Malicious Participants:** The situation becomes more challenging when participants are malicious. A malicious participant may arbitrarily deviate from the prescribed protocol.

For example, during reconstruction, a malicious participant may provide an incorrect value instead of its actual share. The participants may then compute an incorrect value rather than the original secret s .

A malicious participant may also refuse to provide its share. In an n -out-of- n additive sharing scheme, this prevents the remaining participants from reconstructing the secret.

Therefore, malicious participants can affect the *correctness* and *availability* of the protocol even though the underlying additive sharing mechanism provides secrecy against an unauthorized collection of shares.

This illustrates an important limitation of basic additive secret sharing. It provides information-theoretic secrecy of the shares, but by itself it does not ensure that participants use their shares correctly or honestly.

Consequently, when participants may be malicious, additional mechanisms are required to detect incorrect shares, verify consistency, or tolerate participant failures. The specific mechanisms depend on the desired security model and protocol.

4 Computing over Shared Secrets

Secret sharing allows a secret to be distributed among multiple parties such that no individual party learns the secret itself. A natural next question is whether the parties can perform computations directly on shared secrets, without reconstructing the secrets.

Suppose k secrets $s_1, s_2, \dots, s_k$ are shared among a set of parties, and let the desired output be

$$s = f(s_1, s_2, \dots, s_k).$$

The objective of computing over shared secrets is to enable the parties to obtain shares of the new secret s , while keeping the original secrets $s_1, s_2, \dots, s_k$ hidden throughout the computation.

A trivial approach is to reconstruct $s_1, s_2, \dots, s_k$, compute $s = f(s_1, s_2, \dots, s_k)$, and then distribute shares of the resulting secret s . Equivalently, a dealer who knows the secrets could directly generate and distribute shares of s . However, these approaches require additional communication for reconstructing or re-sharing the secrets. More importantly, reconstructing the secrets defeats the purpose of computing on shared data.

Therefore, the central question is:

Can the parties compute shares of $s = f(s_1, s_2, \dots, s_k)$ directly from their existing shares, without reconstructing the underlying secrets or re-distributing new shares?

The answer depends on both the secret-sharing scheme and the function f . For some operations, the new shares can be computed locally from the existing shares. For other operations, additional interaction between the parties is required.

4.1 Computing Addition over Additive Shares

Consider an (n, n) additive secret-sharing scheme over a field $\mathbb{F}$. Let s_1 and s_2 be two secrets. Suppose P_i holds shares $x_i^{(1)}$ and $x_i^{(2)}$ of s_1 and s_2 , respectively. Thus, the shares satisfy $\sum_{i=1}^n x_i^{(1)} = s_1$ and $\sum_{i=1}^n x_i^{(2)} = s_2$.

We want to compute shares of the new secret $s = s_1 + s_2$ without reconstructing either s_1 or s_2 .

Each party can simply add its corresponding shares locally. Party P_i computes

$$z_i = x_i^{(1)} + x_i^{(2)}.$$

The values $z_1, z_2, \dots, z_n$ are then taken as the shares of the new secret s .

4.1.1 Correctness

To show correctness, consider the reconstruction of the newly generated shares. We have

$$\sum_{i=1}^n z_i = \sum_{i=1}^n (x_i^{(1)} + x_i^{(2)}) = \sum_{i=1}^n x_i^{(1)} + \sum_{i=1}^n x_i^{(2)} = s_1 + s_2 = s.$$

Therefore, the values $z_1, z_2, \dots, z_n$ form a valid additive sharing of $s = s_1 + s_2$.

Hence, addition can be performed entirely locally on additive shares:

$$\text{Share}(s_1) + \text{Share}(s_2) = \text{Share}(s_1 + s_2).$$

No communication or re-distribution of shares is required.

4.2 Computing Multiplication over Additive Shares

Consider again an (n, n) additive secret-sharing scheme over a field $\mathbb{F}$. Let s_1 and s_2 be two secrets, where P_i holds shares $x_i^{(1)}$ and $x_i^{(2)}$ such that $\sum_{i=1}^n x_i^{(1)} = s_1$ and $\sum_{i=1}^n x_i^{(2)} = s_2$.

Suppose we want to compute shares of

$$s = s_1 s_2.$$

Unlike addition, multiplication cannot be performed by simply multiplying the corresponding shares. A natural attempt is to let each party compute $z_i = x_i^{(1)} x_i^{(2)}$. However, in general,

$$\sum_{i=1}^n z_i = \sum_{i=1}^n x_i^{(1)} x_i^{(2)} \neq s_1 s_2.$$

The product of the reconstructed secrets contains cross terms involving shares held by different parties. In fact,

$$s_1 s_2 = \left(\sum_{i=1}^n x_i^{(1)} \right) \left(\sum_{j=1}^n x_j^{(2)} \right) = \sum_{i=1}^n \sum_{j=1}^n x_i^{(1)} x_j^{(2)}.$$

Thus, local multiplication of corresponding shares does not produce a valid sharing of the product.

This raises an important distinction between linear and non-linear computation over secret shares. Addition is a linear operation and can be performed locally, whereas multiplication is a non-linear operation and generally requires additional interaction or preprocessed correlated randomness.

4.3 Computing Addition over Shamir Shares

Consider a (t, n) threshold secret-sharing scheme over a finite field $\mathbb{F}_q$. Let s_1 and s_2 be two secrets. They are shared using two independent polynomials

$$p_1(X) = s_1 + a_1 X + \dots + a_{t-1} X^{t-1}$$

and

$$p_2(X) = s_2 + b_1 X + \dots + b_{t-1} X^{t-1},$$

where the coefficients are chosen uniformly at random from $\mathbb{F}_q$.

For each participant P_i , a public and distinct evaluation point $x_i \in \mathbb{F}_q$ is chosen. The share of s_1 held by P_i is the pair (x_i, y_i) , where

$$y_i = p_1(x_i).$$

Similarly, the share of s_2 held by P_i is (x_i, z_i) , where

$$z_i = p_2(x_i).$$

Thus, each participant holds the pair of shares

$$(x_i, y_i), \quad (x_i, z_i).$$

We want to compute a sharing of the new secret

$$s = s_1 + s_2$$

without reconstructing either s_1 or s_2 .

Each participant can locally compute

$$w_i = y_i + z_i.$$

The new share held by P_i is therefore

$$(x_i, w_i).$$

To see why these are valid shares, define the polynomial

$$p(X) = p_1(X) + p_2(X).$$

Then

$$p(X) = (s_1 + s_2) + (a_1 + b_1)X + \cdots + (a_{t-1} + b_{t-1})X^{t-1}.$$

Hence, $p(X)$ has degree at most $t - 1$ and its constant term is $s_1 + s_2$.

Furthermore,

$$w_i = y_i + z_i = p_1(x_i) + p_2(x_i) = p(x_i).$$

Therefore, (x_i, w_i) is precisely the Shamir share corresponding to the polynomial $p(X)$.

4.3.1 Correctness

The constant term of the new polynomial is

$$p(0) = p_1(0) + p_2(0) = s_1 + s_2 = s.$$

Since $p(X)$ has degree at most $t - 1$, any t participants can interpolate $p(X)$. Evaluating the interpolated polynomial at 0 gives

$$p(0) = s.$$

Thus, the locally generated shares (x_i, w_i) form a valid (t, n) Shamir sharing of $s = s_1 + s_2$.

Therefore, addition can be performed directly on Shamir shares without reconstructing the underlying secrets or redistributing new shares.

4.4 Why Multiplication Does Not Work Locally

Consider two secrets s_1 and s_2 shared using Shamir's (t, n) secret-sharing scheme. Let P_i hold (x_i, y_i) and (x_i, z_i) , where y_i and z_i are evaluations of the respective sharing polynomials at x_i .

Suppose the parties want to compute a sharing of $s_1 s_2$. A natural approach is for each participant to locally multiply its shares and set

$$w_i = y_i z_i.$$

The values w_i are evaluations of the polynomial obtained by multiplying the two sharing polynomials. Its constant term is $s_1 s_2$, so the desired secret is indeed encoded in this polynomial.

However, if the original sharing polynomials have degree at most $t - 1$, their product can have degree as high as $2(t - 1)$. Therefore, the resulting values do not form a valid (t, n) Shamir sharing, which requires a polynomial of degree at most $t - 1$.

Thus, unlike addition, local multiplication increases the degree of the sharing polynomial. The parties must perform an additional degree-reduction procedure to obtain a valid (t, n) sharing of the product. Such degree reduction generally requires communication among the parties.

Hence, addition can be performed locally on Shamir shares, whereas multiplication requires additional interaction if the same threshold-sharing structure is to be maintained.

5 Homework

1. In the above construction, the two secrets s_1 and s_2 use the same evaluation points $x_1, \dots, x_n$. Consider the case where the shares of s_1 and s_2 are generated using different evaluation points.

Specifically, suppose participant P_i holds (x_i, y_i) for s_1 , where $y_i = p_1(x_i)$, and (x'_i, z_i) for s_2 , where $z_i = p_2(x'_i)$.

- (a) Can the participants directly compute a valid Shamir sharing of $s_1 + s_2$ by locally adding y_i and z_i ?
- (b) If not, explain why the construction from the previous subsection does not directly work.
- (c) Is there a way to obtain a sharing of $s_1 + s_2$ without reconstructing either secret?